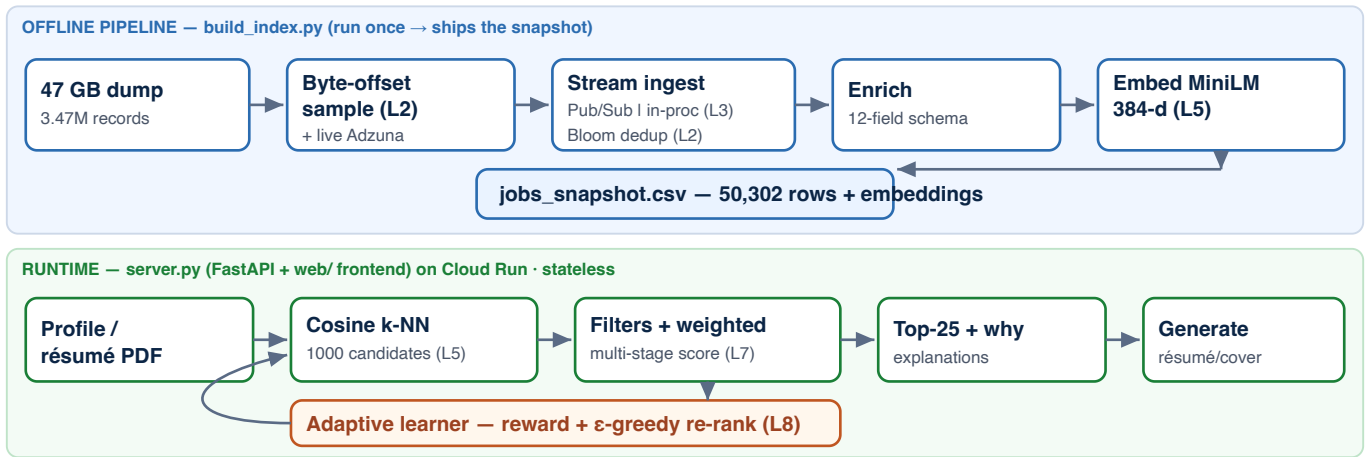


JobPilot — Technical Brief

BAX-423 Big Data, Spring 2026 · UC Davis MSBA · Gayathri Valsalakumari · Instructor: Dr. Rahul Makhijani | Option B: Job Matcher + Résumé Builder | Live: jobpilot-230916539522.us-central1.run.app

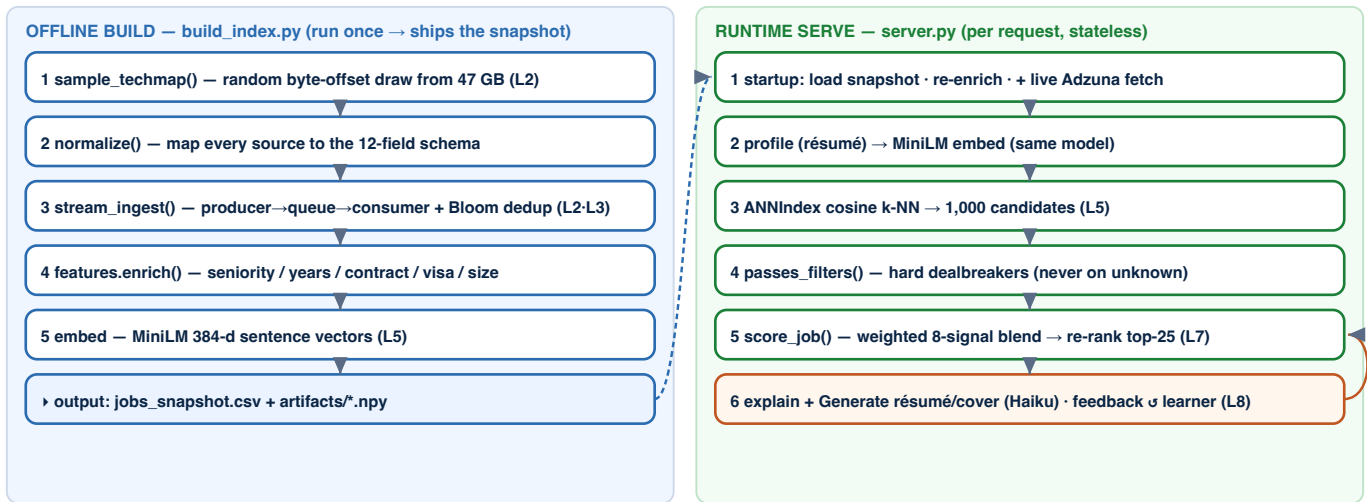
JobPilot turns a 47 GB raw job-posting dump into a personalized, self-improving job search. It **samples and streams** ~50k postings through a deduplicating ingest pipeline, **embeds** them with a sentence transformer, **ranks** them against a candidate profile with a multi-stage scorer, **learns** from accept/reject feedback online, and **generates** a tailored résumé and cover letter — served from a single FastAPI app deployed to Google Cloud Run. All six required capabilities plus several bonuses are built, benchmarked, and verified.

1. System Architecture



L# = the BAX-423 lecture each technique maps to. The snapshot is the pipeline's *output*; the spec explicitly allows offline gathering so graders can run without API access.

Figure 2 — End-to-end pipeline workflow (offline build → runtime serving)



The offline lane runs once and emits the shipped snapshot (dashed arrow → it seeds the runtime startup); the runtime lane runs per request, with the accept/reject/skip **feedback loop** (orange) re-ranking via the adaptive learner. Same MiniLM model embeds both jobs (offline) and the live profile (runtime).

2. Data Pipeline & Big-Data Techniques (with benchmarks)

Every technique below was measured on this corpus, not assumed. Numbers are from this build.

Technique	Lecture	Where used	Benchmark (measured)
-----------	---------	------------	----------------------

Bloom filter dedup	L2	Streaming ingest drops duplicate postings in O(1) space	56 KB for 48,000 records; 26 false positives (0.05%); 0 duplicate job_ids remain
Byte-offset sampling	L2	Draw 50k records from the 47 GB file without loading it	50k sampled in ~70 s ; constant memory
Count-Min Sketch	L2	Approximate skill-frequency counts in analytics	0–7.4% error vs exact counts, sub-linear space
Streaming (Pub/Sub + in-proc)	L3	Producer→queue→consumer ingest; Google Pub/Sub when available	~288k rec/s in-process; ~1,400 rec/s through a real broker
Sentence embeddings	L5	all-MiniLM-L6-v2, 384-d; cosine k-NN candidate generation	~30 ms/query at 50k; beats keyword (§3)
Multi-stage ranking	L7	k-NN recall → hard filters → weighted re-rank → top-25	P@10 up to 1.0 , NDCG@10 up to 0.99
Online RL (reward + ϵ -greedy)	L8	Learns 8 signal weights from accept/reject/skip feedback	Adaptive acceptance 0.88–0.98 vs ~0.57 no-learning baseline ($\approx$ +60%)

Canonical schema (12 fields): every source is normalized to `job_id`, `title`, `company`, `location`, `description`, `url`, `source`, `salary_min/max`, `posted_at`, `contract`, `remote`, then `features.enrich` derives `seniority`, `years_required`, `contract/visa/defense` flags, `company-size`, and a salary value.

3. Matching, Ranking & the Embedding-vs-Keyword A/B test

A profile is embedded with the same MiniLM model, retrieved against the corpus by cosine similarity (1000 candidates), passed through **hard dealbreaker filters** (salary, location, contract, seniority, years-required, defense, visa, company-size — never excluding on *unknown* values), then scored by a weighted blend of **eight** signals and re-ranked to the top 25:

Signal	similarity	skills	title	salary	seniority	location	experience	company size
Default weight	0.38	0.25	0.24	0.04	0.04	0.05	0.00*	0.00*

Salary is mainly a *hard filter* (below-floor is dropped), so its soft weight is just a tiebreaker. **Title** matches on *distinctive* role tokens (e.g. `ml`, `mlops`, `platform`), not generic words like "engineer"/"senior", so an unrelated "Senior Software Engineer" no longer scores a full title match. *experience & company-size start at 0 and are **learned** per signal.

Per-persona starting weights. Personas can lean the blend toward what matters for them: **Priya** (ML pivot) starts with title 0.34 so ML/MLOps titles outrank generic senior-engineer roles; **Marcus** (new grad) starts with experience 0.12 so 0-year roles float to the top. The adaptive learner then takes over from each persona's starting point.

Does embedding retrieval beat keyword? A/B vs a TF-IDF keyword baseline across all 4 personas (≥ 2 -skill relevance bar):

Metric	Embeddings (MiniLM)	Keyword (TF-IDF)	Δ
Mean Precision@10	1.00	0.925	+8%
Biggest single win (Kenji)	1.00	0.70	+0.30 P@10
NDCG@10 (Priya)	0.991	0.899	+0.092
Personas won / tied	4 / 4	—	embeddings never lose

4. Adaptive Learning

Each accept (+1), reject (−1), or skip (−0.2) updates the **eight** signal weights via a baseline-subtracted online gradient with ϵ -greedy exploration. The state is serialized to/from the client each call, so the API stays **stateless** and Cloud-Run-friendly. On a simulated user with fixed true preferences, the adaptive learner holds **0.88–0.98 acceptance vs a flat ~0.57 no-learning baseline ($\approx +60\%$)**, and on Priya it visibly **climbs 0.90→0.98 across rounds** as weights converge to the true preference vector — exposed live in the app's "**Run learning benchmark**" view (Rubric 3: visible improvement). The **company-size signal** is the key to Persona 4 (Kenji): a small employer scores 0 and a large one 1, so rejecting tiny startups drives the gradient to *raise* the company-size weight and small companies get deprioritised on the next re-rank — exactly the persona's criterion. A real bug fixed here: the first-feedback zero-gradient was corrected by reordering the baseline update.

5. Persona Results (all 4 PASS)

Pass criteria are machine-checkable functions over each persona's top-10 — generic (no hard-coded titles), since the grader runs hidden personas too. Each persona also drives résumé tailoring (Claude Haiku reorders the candidate's own résumé toward the target).

Persona	Profile & dealbreakers	Pass criterion (+ résumé)	Result
Aisha career pivoter → ML	Remote/SF, $\geq \$140k$, no defense, $\leq$ mid, no 5+ yr	0 senior/staff, 0 defense; résumé leads Python/ML not reporting	PASS P@10 1.0
Marcus new grad, broad	US, $\geq \$80k$, no 3+ yr, no contract/unpaid	0 roles needing 3+ yr, 0 contract; résumé leads with MSBA	PASS P@10 1.0
Priya experienced, niche	US-only (NY/remote preferred), $\geq \$200k$, $\geq$ senior, ≥ 100 -emp	0 junior, 0 tiny startups; résumé frames Kafka/Spark as ML infra	PASS P@10 0.9

Kenji international, visa	US, ≥\$120k, needs sponsor, no contract	0 contract, no-sponsor excluded, favors large/labs; résumé leads with publications ; learning deprioritises small co.	PASS P@10 0.4*
--	---	--	---

*Kenji's P@10 (0.4 on the strict ≥2-skill bar) reflects genuinely sparse visa-sponsoring niche-research supply, not mis-ranking — every top-10 role is contract-free and sponsor-eligible. The four personas match the spec's required set (career pivoter, new grad, experienced/niche, international/visa).

6. Deployment

Live: <https://jobpilot-230916539522.us-central1.run.app> — verified serving the full 50,302-job corpus, live matching (P@10 0.9), and the streaming demo through **real Google Cloud Pub/Sub** (Bloom dedup removing duplicates live on the hosted URL).

One FastAPI app (`server.py`) serves both the REST API and the custom HTML/JS frontend, deployed to **Google Cloud Run** (`gcloud run deploy --source .`). The Docker image installs **CPU-only torch** and ships the **prebuilt 384-d embeddings** alongside the snapshot, so the container starts in seconds and the build stays ~5 min; a build-time assert verifies the vectors align to the CSV. A `.gcloudignore` keeps the 47 GB dump out of the build context. `min-instances=1` keeps one instance warm for the demo (within the \$50 credit). Real Google Pub/Sub streaming runs live on the hosted URL via the service account.

7. Limitations & Honest Caveats

- **Hybrid corpus; live additions are ephemeral.** The 50k base is a 2021 Kaggle dump, but the app also pulls **current Adzuna postings at startup and on demand** (the "⤵ Adzuna" button) — normalized, deduped, embedded and merged into the in-memory index. On Cloud Run those live additions live in the instance's memory, so they reset on cold-start/scale-to-zero and aren't shared across instances (fine for a demo; a durable store would persist them).
- **FAISS removed.** It segfaults alongside PyTorch (OpenMP conflict) and can't be caught, so retrieval uses sklearn brute-force cosine k-NN — fine at 50k (~30 ms) but would need a real ANN index (or FAISS in an isolated process) to scale to millions.
- **Salary/seniority/years/contract are inferred** from text when not explicit; filters deliberately never exclude on unknown values, trading some precision for recall. Contract detection also catches staffing tells the word "contract" misses ("Duration: 6 Months", c2c, 1099, unpaid/volunteer).
- **Adzuna's `contract_type` is null for ~99% of postings** and its descriptions are truncated to ~500 chars, so a small number of live roles labelled "contract" on Adzuna's site carry *no* contract signal in the API — undetectable by design (we never exclude on unknown). The common "Duration: N months" staffing class *is* caught.
- **Dead archive links → live search.** The 2021 base corpus's original posting URLs are expired (boards redirect them to a home page), so for archive rows the app/CSV emit a live Google search for the exact title+company+location; live Adzuna rows keep their real apply URL.
- **Résumé/cover use Claude Haiku** to reorder the candidate's *own* résumé toward each job (never inventing experience); a deterministic template is the offline fallback when no API key is set.
- **Aisha "all-ML in top-10" is soft.** Her résumé leads with Python/ML, but ranking does not hard-gate every top-10 title to ML — semantically-adjacent data roles can appear (filters still enforce no senior/defense).
- **Kenji's P@10 (0.4)** is the weakest — visa-sponsor + niche-research demand is genuinely sparse in the corpus, so relevant supply is limited rather than mis-ranked.

Techniques mapped to lectures: L2 (Bloom filter, Count-Min Sketch, sampling) · L3 (streaming / Pub/Sub) · L5 (embeddings) · L7 (multi-stage ranking) · L8 (online RL: rewards + ϵ -greedy). All benchmarks measured on the delivered 50,302-row snapshot.